## Problem Statement:

Given a positive integer N, print all the numbers from 1 to N in increasing order, separated by a single space.

Your task is to write a program that prints the required sequence.

After completing the program, analyze your solution by calculating its Time Complexity and Space Complexity, and explain why your answer is correct.

**Input Format**

A single integer N.

**Output Format**

Print all integers from 1 to N in increasing order separated by spaces.

**Constraints**

$1 \leq N \leq 10^5$

**Example 1**

Input

5

Output

1 2 3 4 5

Explanation

The numbers from 1 to 5 are printed in increasing order.

**Example 2**

Input

8

Output

1 2 3 4 5 6 7 8

Explanation

The program prints every integer starting from 1 until 8, maintaining increasing order.

# WEEK 04 MODULE

Q: Problem Statement 01 —
Print numbers from 1 to N.

```
import java.util.Scanner;

class PrintNumber {
    public static void main (String[] args){
        Scanner sc = new Scanner (System.in);
        System.out.print ("Enter a number: ");
        int num = sc.nextInt ();

        for (int i = 1 ; i <= num ; i++) {
            System.out.print (i + " ");
        }

        sc.close ();
    }
}
```

* Time Complexity

Since loop runs from 1 to N thus each iteration performs a constant-time print.

N operations → O(N).

* Space Complexity -

There is only one integer variable N and loop counter.

Space Complexity = O(1)

Complexity is justified with loop count and memory usage reasoning.

File Edit Selection View ··· Week4Module.java

PrintNumber.java

PrintNumber.java > PrintNumber > main(String[])

```java
import java.util.Scanner;
class PrintNumber {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);

        System.out.print(s: "Enter a number:");

        int num=sc.nextInt();

        for(int i=1;i<=num;i++){
        System.out.print(i+" ");
        }

        sc.close();
    }
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\MISTHI\OneDrive\Desktop\college practice module\Week4Modu
howCodeDetailsInExceptionMessages' '-cp' 'C:\Users\MISTHI\AppData\Roa
dhat.java\jdt_ws\Week4Module.java_e1b144d7\bin' 'PrintNumber'
Enter a number:10
1 2 3 4 5 6 7 8 9 10
PS C:\Users\MISTHI\OneDrive\Desktop\college practice module\Week4Modu
```

## Problem Statement:

Given a positive integer N, find the sum of the first N natural numbers using a loop.

Your task is to write a program that calculates and prints the required sum.

After completing the program, analyze your solution by calculating its Time Complexity and Space Complexity, and explain your answer.

**Input Format**

A single integer N.

**Output Format**

Print the sum of the first N natural numbers.

**Constraints**

$1 \leq N \leq 10^7$

**Example 1**

Input

5

Output

15

Explanation

The sum of the first five natural numbers is:

$1 + 2 + 3 + 4 + 5 = 15$

**Example 2**

Input

10

Output

55

Explanation

The sum of the first ten natural numbers is:

$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 = 55$

Q: Problem Statement 02 –

Sum of first N Natural Numbers

```
import java.util.Scanner;

public class Sum {
  public static void main (String[] args){

    Scanner sc = new Scanner (System.in);

    int num = new sc.nextInt ();
    long sum = 0;

    for (int i = 1, i <= num, i++) {
```

```
        sum += i;
    }
    System.out.print("The sum of first" + num + "terms of natural number is " + sum);
    sc.close();
  }
}
```

* Time complexity –

The loop runs from 1 to N.
Thus total operation is N.
Time complexity = O(N)

* Space Complexity.

The program have three variables.
No other data structure or recursion are used.
∴ Space complexity = O(1)

We could have used sum = $\frac{N*(N+1)}{2}$ which decreases time complexity by O(1). Space complexity remains same.

File Edit Selection View ··· Week4Module.java

PrintNumber.java | SumofNaturalNumbers.java

SumofNaturalNumbers.java > SumofNaturalNumbers

```java
import java.util.Scanner;

public class SumofNaturalNumbers {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);

        System.out.print(s: "Enter a number:");
        int num=sc.nextInt();

        long sum=0;
        for(int i=1;i<=num;i++){
            sum+=i;
        }
        System.out.println("Sum of first "+num+" natural numbers is: "+sum);
        sc.close();
    }
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Run: Su

```
PS C:\Users\MISTHI\OneDrive\Desktop\college practice module\Week4Module.java>  & 'C:\Program Files
howCodeDetailsInExceptionMessages' '-cp' 'C:\Users\MISTHI\AppData\Roaming\Code\User\workspaceStora
dhat.java\jdt_ws\Week4Module.java_e1b144d7\bin' 'SumofNaturalNumbers'
Enter a number:5
Sum of first 5 natural numbers is: 15
PS C:\Users\MISTHI\OneDrive\Desktop\college practice module\Week4Module.java>
```

## Problem Statement:

Given two integers A and B, calculate the value of A<sup>B</sup> (A raised to the power B) using a loop.

Your task is to write a program that computes the result and prints it.

Once your program is complete, analyze your solution by calculating its Time Complexity and Space Complexity, and explain why your answer is correct.

**Input Format**

Two integers A and B separated by a space.

**Output Format**

Print the value of A<sup>B</sup>.

**Constraints**

$1 \leq A \leq 100$

$0 \leq B \leq 20$

**Example 1**

Input

2 5

Output

32

Explanation

The value of 2<sup>5</sup> is:

$2 \times 2 \times 2 \times 2 \times 2 = 32$

**Example 2**

Input

3 4

Output

81

Explanation

The value of 3<sup>4</sup> is:

$3 \times 3 \times 3 \times 3 = 81$

Q: Problem Statement 03 –

Compute $A^B$ using a loop

```
import java.util.Scanner;

public class CalculatePower {
  public static void main (String[] args) {

    Scanner sc = new Scanner (System.in);

    int x = sc.nextInt ();
    int y = sc.nextInt ();

    long power = 1;

    for (int i = 1; i <= y; i++) {
        power *= x;
    }

    System.out.println (power);
    sc.close ();
  }
}
```

* Time Complexity -
The loop runs y time then time complexity is O(y).

* Space Complexity
Three variables (x, y and power) is used in the program. There is no extra data structure or recursion. Thus, space complexity is O(1).

File Edit Selection View ··· Week4Module.java

PrintNumber.java | SumofNaturalNumbers.java | CalculatePower.java

CalculatePower.java > CalculatePower

```java
public class CalculatePower {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);

        System.out.print(s: "Enter the two numbers with space between them:");
        int x=sc.nextInt();
        int y=sc.nextInt();

        long result=1;

        for(int i=1;i<=y;i++){
            result*=x;
        }
        System.out.println(result);
        sc.close();
    }
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\MISTHI\OneDrive\Desktop\college practice module\Week4Module.java>  & 'C:\Program File
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\MISTHI\AppData\Roaming\Code\User\worksp
30893397c88\redhat.java\jdt_ws\Week4Module.java_e1b144d7\bin' 'CalculatePower'
Enter the two numbers with space between them:2 5
32
PS C:\Users\MISTHI\OneDrive\Desktop\college practice module\Week4Module.java>
```

## Section A – Complexity Analysis of Previous Modules:

In this section, revisit all the topics and programming problems covered in Module 1, Module 2, and Module 3.
For each topic and question, write:

- Time Complexity
- Space Complexity
- A brief explanation (2–3 lines) justifying why the complexity is correct.
- If an optimized approach exists, mention it.

**Note:** Do not simply write O(n) or O(1). Your explanation should clearly describe how the algorithm works and why it results in that complexity.

# Week 04 Module

## ⁂ Section A

### 1. Square Pattern

* Time Complexity : $O(n^2)$
There are two nested loops, outer loop runs n times and inner loop also runs n times.

* Space Complexity : $O(1)$
Only loop counters are used, Only two variable (x, y) are used and there is no other data structure or recursion.

### 2. Hollow Square Pattern

* Time Complexity : $O(n^2)$
Two nested loops are used.

* Space Complexity : $O(1)$
Two variables are used and there is no other data structure or recursion.

## 5. Right Angled Triangle

* Time Complexity : $O(n^2)$
The outer loop runs n times and the inner loop runs upto current row index.
* Space Complexity : $O(1)$
No extra data structure or recursion used.

## 4. Inverted Triangle Pattern

* Time Complexity : $O(n^2)$
Two loops are running. One runs n times while other prints ~~for~~ ~~stars~~ & spaces.
* Space Complexity : $O(1)$
No extra data structure or recursion used.

5. Pyramid

* Time Complexity : $O(n^2)$
  Two loops are used.
* Space Complexity : $O(1)$
  No recursion or ds used.

6. Half Diamond / Half Diamond Inverted Pattern

* Time Complexity : $O(n^2)$
  Two loops used.
* Space Complexity : $O(1)$
  No recursion or data structured used.

7. Number Patterns / Incrementing No. / Right Triangle / Inverted Diamond / Sandwich Pattern / Incrementing Diamond Pattern.

* Time Complexity : $O(n^2)$
  All these patterns have 2 loops working.
* Space Complexity : $O(1)$
  All these pattern have variables but no recursion or no data structure

## ☆2 Module 2 Analyses

1. Fraud Transaction Detection
   Works on Linear Search

* Time Complexity
  Best case - O(1) [Suspicious ID is found at first place.]
  Worst case. - O(N) [Suspicious ID is found at last place]
  Average Case - O(N)
  The algo checks each transaction sequentially until it finds the correct one.

* Space Complexity
  Only one array of size N and a few variables are used, no extra storage.

* Optimization
  ~~If~~ Binary search ~~can~~ used if the transactions ~~were~~ sorted.
  It could reduce time ~~to~~ to O(log N).

2. Server Log Search.
↳ works on binary search

* Time Complexity
Best case → O(1) [key at middle]
Worst Case → O(logN)
Average Case → O(logN)
Each iteration halves the search space, so the no. of comparisons grows logarithmically.

* Space Complexity
Only variables for index and ~~key~~ are used, thus it is O(1).

3. Sort Employee Salary
↳ Bubble Sort

* Time Complexity
Best → O(N)
Worst → O(N²)
Avg. → O(N²)
Nested loops compare and swap adjacent elements repeatedly.

* Space Complexity
No extra memory required, thus O(1)

* Optimization
Merge Sort or Quick Sort can be used.

4. Sort Website Response Time
↳ works on Insertion Sort

* Time Complexity
Best - O(N) [Already sorted]
Worst - $O(N^2)$
Average - $O(N^2)$
Each new element is compared with previous ones and inserted in correct position.

* Space Complexity - O(1)
No extra memory is used.

* Optimization
Merge Sort or Heap Sort can be used for datasets.

5. Prioritize Bug Reports
↳ Works on Selection Sort

* Time Complexity
Best - $O(N^2)$
Worst - $O(N^2)$
Average - $O(N^2)$
For each position, the algorithm scans the remaining array to find the minimum & swaps it.

* Space Complexity - $O(1)$
Sorting is in-place, no extra storage is required.

* ~~Optimization~~ -
Heap Sort or Merge Sort can be used.

# ☆ Module 3 Analysis –

## 1. Sort Salary Packages.

* Time Complexity
  O(N Log N)
  It divides each array into half and merges each level with N operations.
* Space Complexity
  O(N)
  Extra temporary array is required during merging.
* Optimization
  For small arrays, insertion sort is faster.

## 2. Processing Time Analysis

* Time Complexity
  O(N Log N) for sorting
  O(N) for median, count, differ...
* Space Complexity
  O(N)
  Temporary array is used.

3. Server Response Time.

* Time Complexity
Best - O(NlogN)
worst - $O(N^2)$
Average - O(NlogN)
It partitions the array around a pivot and recursively sorts subarrays

* Space Complexity - O(logN)
Due to recursion.

4. Trade Value Analysis

* Time Complexity
O(NlogN) for sorting
O(N) for statistics.

* Space Complexity
O(logN)
for recursion stack.

5. Player Scores.

* Time Complexity
  O(N log N)
  It repeatedly extracts max/min for heap, each extraction costs log N.

* Space Complexity
  O(1)
  Sorting is in place.

6. Emergency Response Time Analysis.

* Time Complexity
  O(N log N)

* Space Complexity
  O(1)
  In-place sorting.